Scheme of Valuation/Answer Key

(Scheme of evaluation (marks in brackets) and answers of problems/key)

APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY

SECOND SEMESTER B.TECH DEGREE EXAMINATION(2019 SCHEME), JULY 2021

Course Code: EST 102

Course Name: PROGRAMMING IN C (Common to all programs)

Max. Marks: 100	**(FN Session)**	Duration: 3 Hours

PART A

		Answer all Questions. Each question carries 3 Marks	Marks
1		Differences -3Marks. 1. System Software is the type of software which is the interface between application software and system. On other hand Application Software is the type of software which runs as per user request. It runs on the platform which is provide by system software. 2. System software is used for operating computer hardware. On other hand Application software is used by user to perform specific task. 3. Some examples of system software's are compiler, assembler, debugger, driver, etc. Some examples of application software's are word processor, web browser, media player, etc.	(3)
2		Difference – 3Marks **Interpreter** translates just one statement of the program at a time into machine code. **Compiler** scans the entire program and translates the whole of it into machine code at once. Interpreter is slow where as compilers are fast translators.	(3)
3		Precedence-1Mark Associativity-1Mark Table-1 Mark **Precedence**: specifies the order of evaluation of operators in an expression. **Associativity**: It defines the order in which operators of the same precedence are evaluated in an expression. Associativity can be either from left to right or right to left.	(3)
4		Difference using a sample program fragment- 3 Marks. The main difference between break and continue is that break is used for immediate termination of loop. On the other hand, 'continue' terminate the current iteration and resumes the control to the next iteration of the loop.	(3)

5		Any 3 string handling functions-0.5 Mark each. Eg:strcat(), strlen(), strcpy() Examples- 0.5Mark each.	(3)

6		Program conveying correct logic -3 Marks.
		Sample Pgm:
		#include <stdio.h>
		
		int main ()
		{
		int arr[100], freq[15];
		int size, i, j, count;
		printf("Enter size of array: ");
		scanf("%d", &size);
		printf("Enter elements in array: ");
		for(i=0; i<size; i++)
		{
		scanf("%d", &arr[i]);
		
		/* Initially initialize frequencies to -1 */
		freq[i] = -1;
		}
		
		for(i=0; i<size; i++)
		{
		count = 1;
		for(j=i+1; j<size; j++)
		{
		/* If duplicate element is found */
		if(arr[i]==arr[j])
		{
		count++;
		
		/* Make sure not to count frequency of same element again */
		freq[j] = 0;
		}
		}
		
		/* If frequency of current element is not counted */
		if(freq[i] != 0)
		{
		freq[i] = count;
		}
		}
		
		/*
		
		printf("\nFrequency of all elements of array : \n");
		for(i=0; i<size; i++)
		{
		if(freq[i] != 0)
			(3)

		{
		printf("%d occurs %d times\n", arr[i], freq[i]);
		}
		}
		
		return 0;
		}	

7		Formal parameters-1Mark Actual parameter-1Mark. Example- 1Mark.
		Actual Parameters are the values that are passed to the function when it is invoked while Formal Parameters are the variables defined by the function that receives values when the function is called.
		int sum(int a, int b) //Statement 1
		{
		return a+b;
		}
		int main()
		{
		int x=10,y=20;
		int s = sum(x,y); //Statement 2
		return 0;
		}
		In *Statement 1*, the variables a and b are called FORMAL PARAMETERS.
		In *Statement 2* the arguments x and y are called ACTUAL PARAMETERS
			(3)

8		Sample illustrations-1.5 marks each	(3)
9		Each function – 1 mark	(3)
10		Address operator(&) and indirection(*) operator- 1.5 marks each The operator & and the immediately preceding variable returns the address of the variable associated with it. Indirection operator also called as value at address. It returns a value stored at that address.	(3)
		PART B	
		Answer any one Question from each module. Each question carries 14 Marks	
11	a)	Detailed explanation -7marks	(7)
	b)	Correct algoritm-7 marks Sample: 1. Input a Number 2. Initialize Sum to zero 3. While Number is not zero 4. Get Remainder by Number Mod 10 5. Add Remainder to Sum 6. Divide Number by 10 7. Print sum	(7)

		OR	
12	a)	Explanation - 4 Marks Flowchart - 4 Marks Algorithm- 6Marks	(14)
13	a)	**Any 4 data types with explanation** - 4 marks. **Memory requirements**- 3 marks. int or signed int-2bytes Integers are whole numbers that can have both zero, positive and negative values but no decimal values. Float -4 bytes Float and double are used to hold real numbers. The floating point data type allows the user to type decimal values. For example, average marks can be 97.665. if we use int data type, it will strip off the decimal part and print only 97. To print the exact value, we need 'float' data type. Char-1 byte char stores a single character. Char consists of a single byte. Double-8 bytes Double is 8 bytes, which means you can have more precision than float. This is useful in scientific programs that require precision. Float is just a single-precision data type; double is the double-precision data type.	(7)
	b)	Program conveying correct logic-7 Marks	(7)
		OR	

14	a)	Correct program-7 Marks
		#include <stdio.h>
		
		int main ()
		{
		int num, sum=0, firstDigit, lastDigit;
		printf ("Enter any number to find sum of first and last digit: ");
		scanf("%d", &num);
		
		/* Find last digit to sum */
		lastDigit = num % 10;
		
		/* Copy num to first digit */
		firstDigit = num;
			(7)

		/* Find the first digit by dividing num by 10 until first digit is left */
		while (num >= 10)
		{
		num = num / 10;
		}
		firstDigit = num;
		
		/* Find sum of first and last digit*/
		sum = firstDigit + lastDigit;
		printf("Sum of first and last digit = %d", sum);
		return 0;
		}	

	b)	Explanation – 2Marks. Any two functions- 2 marks. Difference 3 marks Typecasting refers to the conversion of one data type to another by the user, whereas type conversion refers to the automatic conversion of one type of data to another. atoi() function in C language converts string data type to int data type. itoa () function in C language converts int data type to string data type. The basic difference between type conversion and type casting, i.e. type conversion is made "automatically" by compiler whereas, type casting is to be "explicitly done" by the programmer. ... When the two types are compatible with each other, then the conversion of one type to other is done automatically by the compiler.	(7)

15	a)	Correct program-7 Marks
		#include <stdio.h>
		int main()
		{
		int array[100], search, c, n;
		printf("Enter number of elements in array\n");
		scanf("%d", &n);
		printf("Enter %d integer(s)\n", n);
		for (c = 0; c < n; c++)
		scanf("%d", &array[c]);
		printf("Enter a number to search\n");
		scanf("%d", &search);
		for (c = 0; c < n; c++)
		{
		if (array[c] == search) /* If required element is found */
		{
		printf("%d is present at location %d.\n", search, c+1);
		break;
		}
		}
		if (c == n)
		printf("%d isn't present in the array.\n", search);
		return 0;
		}
			(7)

	b)	Correct program-7 Marks	(7)

```
#include <stdio.h>
#include <string.h>

int main()
{
        char Str[100], RevStr[100];
        int i, j, len;

        printf("\n Please Enter any String :  ");
        gets(Str);

        j = 0;
        len = strlen(Str);

        for (i = len - 1; i >= 0; i--)
        {
                RevStr[j++] = Str[i];
        }
        RevStr[i] = '\0';

        printf("\n String after Reversing = %s", RevStr);

        return 0;
}
```

OR

16 a) Correct program-7 Marks (7)

```
#include <stdio.h>

int main()
{
 int m, n, c, d, matrix[10][10], transpose[10][10];

 printf("Enter the number of rows and columns of a matrix\n");
 scanf("%d%d", &m, &n);
 printf("Enter elements of the matrix\n");

 for (c = 0; c < m; c++)
  for (d = 0; d < n; d++)
   scanf("%d", &matrix[c][d]);

 for (c = 0; c < m; c++)
  for (d = 0; d < n; d++)
   transpose[d][c] = matrix[c][d];

 printf("Transpose of the matrix:\n");

 for (c = 0; c < n; c++)
{
  for (d = 0; d < m; d++)
   printf("%d\t", transpose[c][d]);
```

<table>
<tr><td></td><td></td><td>
<pre>
  printf("\n");
 }
 return 0;
}
</pre>
</td><td></td></tr>
<tr><td></td><td>b)</td><td>Correct program-7 Marks
<pre>
#include <stdio.h>

int main()
{
        char str[100];
        int i, vowels, consonants;
        i = vowels = consonants = 0;

        printf("\n Please Enter any String : ");
        gets(str);

        while (str[i] != '\0')
        {
                if(str[i] == 'a' || str[i] == 'e' || str[i] == 'i' || str[i] == 'o' || str[i] == 'u' ||
                str[i] == 'A' || str[i] == 'E' || str[i] == 'I' || str[i] == 'O' || str[i] == 'U')
                {
                        vowels++;
                }
                else
                consonants++;
        i++;
        }
  printf("\n Number of Vowels in this String = %d", vowels);
        printf("\n Number of Consonants in this String = %d", consonants);

        return 0;
}
</pre>
</td><td>(7)</td></tr>
<tr><td>17</td><td>a)</td><td>function declaration and function definition – 1.5 marks each
function call – 1 mark
Sample program illustrating syntax- 3 Marks</td><td>(7)</td></tr>
<tr><td></td><td>b)</td><td>Structure Creation -3Marks
Function to read data- 2 Marks
Function to display data- 2 Marks.</td><td>(7)</td></tr>
<tr><td></td><td></td><td>OR</td><td></td></tr>
<tr><td>18</td><td>a)</td><td>Storage class definition -1 mark.
Any 3 storage classes (Auto, extern, static, register)- 1 mark each.
Example – 1 Mark each</td><td>(7)</td></tr>
<tr><td></td><td>b)</td><td>Correct program using functions-7 Marks</td><td>(7)</td></tr>
</table>

19	a)	Minimum 4 modes to be explained. **"r"(read) –** Searches file. If the file is opened successfully fopen( ) loads it into memory and sets up a pointer which points to the first character in it. If the file cannot be opened fopen( ) returns NULL. **"w"(write) –** Searches file. If the file exists, its contents are overwritten. If the file doesn't exist, a new file is created. Returns NULL, if unable to open file. **"a"(append) –** Searches file. If the file is opened successfully fopen( ) loads it into memory and sets up a pointer that points to the last character in it. If the file doesn't exist, a new file is created. Returns NULL, if unable to open file. **"r+" –** Searches file. If is opened successfully fopen( ) loads it into memory and sets up a pointer which points to the first character in it. Returns NULL, if unable to open the file. **"w+" –** Searches file. If the file exists, its contents are overwritten. If the file doesn't exist a new file is created. Returns NULL, if unable to open file. **"a+" –** Searches file. If the file is opened successfully fopen( ) loads it into memory and sets up a pointer which points to the last character in it. If the file doesn't exist, a new file is created. Returns NULL, if unable to open file.	(7)
	b)	Correct program-7 Marks	(7)
		OR	

20	a)	Explanation with a sample program- 4+3 marks
		When we pass a pointer as an argument instead of a variable then the address of the variable is passed instead of the value. So any changes made by the function using the pointer is permanently made at the address of passed variable.
		`/* Pass Pointers to Functions in C Sample Program */`
		
		`#include<stdio.h>`
		
		`void Swap(int *x, int *y)`
		`{`
		`  int Temp;`
		`  Temp = *x;`
		`  *x = *y;`
		`  *y = Temp;`
		`}`
		
		`int main()`
			(7)

		{
		int a, b;
		
		printf ("Please Enter 2 Integer Values : ");
		scanf("%d %d", &a, &b);
		
		printf("\nBefore Swapping A = %d and B = %d", a, b);
		
		Swap(&a, &b);
		
		printf(" \nAfter Swapping A = %d and B = %d \n", a, b);
		}	

	b)	Explanation of any 5 functions with syntax- 7 marks fopen() fgets() fprintf() fclose() fseek()	(7)
NOTE: Marks may be awarded for programs conveying correct logic. Sample programs are included in scheme.			